Rapport — tests de charge (catalogue recruteur)

Destinataire Thomas Vignal, conseiller numérique
Référence JEB/DNI/2026-003 · courriel du 8 septembre 2026 (« Tests de charge, c'est maintenant »)
Échéance jeudi, avant la revue technique
Rédacteur Victor — infrastructure / tests de charge

Ce document répond au périmètre minimal demandé : 500 profils dont 300+ avec vidéo, 100 utilisateurs simultanés parcourant le catalogue, temps de réponse médian et p95 sur les trois routes les plus sollicitées, nombre d'erreurs, goulot d'étranglement identifié. La sortie brute de l'outil est jointe (§6), pas seulement cette synthèse.

---

1. Méthode

Outil	k6 v2.2.0 — binaire unique, scénario versionné dans `load-test/`
Scénario	`load-test/catalogue.js` : un recruteur pagine le catalogue (parfois filtré par secteur/localisation, parfois trié par score), puis ouvre le profil et les vidéos d'une partie des candidats consultés. 0,5-2 s de réflexion entre deux requêtes — un recruteur qui clique, pas un script qui martèle.
Jeu de données	500 profils, 300 vidéos (`npm run db:seed -w @profils-actifs/db` , `packages/db/src/seed-data/generated-applicants.ts`)
Machine	AMD Ryzen 5 7520U (8 threads), 14 Gio RAM — poste de développement partagé, pas un serveur dédié
Date	8 septembre 2026, ~14h15-14h30 (heure de Paris)
API	build de production (`npm run build -w @profils-actifs/api` puis `node dist/index.js` , `NODE_ENV=production`)
PostgreSQL	16.15, Docker (`docker-compose.yml`)

Le rate limiter de l'API (100 requêtes / 15 min par IP, `packages/api/src/middleware/rate-limit.ts`) a été relevé (`RATE_LIMIT_MAX_REQUESTS`) pour la durée des mesures — chaque VU k6 partage la même IP sur cette machine, et le limiter est fait pour bloquer exactement ce trafic sinon. Détail et procédure dans `load-test/README.md` .

2. Résultat — 100 recruteurs simultanés, rythme réaliste

`k6 run load-test/catalogue.js` (100 VU, 2 min) :

Route	Médiane	p95	Erreurs
`GET /api/recruiter/catalogue`	20,5 ms	38,0 ms	0
`GET /api/profiles/:id`	8,9 ms	16,7 ms	181 / 1570 (11,5 %)
`GET /api/profiles/:id/videos`	5,8 ms	12,3 ms	0

6968 requêtes, 55,5 req/s, 4007 itérations complètes, 0 timeout, 0 réponse 5xx. Sortie brute : `load-test/results/baseline-2026-09-08.txt` / `.json`.

Sur les 181 erreurs, aucune n'est un problème de performance. Ce sont toutes des 404 sur `GET /api/profiles/:id`, et la cause est connue et documentée (`load-test/README.md`, section « Known issue ») : le catalogue recruteur liste les profils `APPROVED` et `PENDING`, mais la page profil publique (`isPubliclyVisible`, `packages/api/src/services/visibility.ts`) n'autorise que `APPROVED`. Un recruteur voit un candidat `PENDING` dans son catalogue, clique, et tombe sur « profil introuvable » — reproductible à 1 VU, sans lien avec la charge. La vraie application a le même comportement (`ProfileCard.tsx:107` pointe vers la même route). Signalé séparément comme un défaut de visibilité à corriger ; pas un résultat de ce test de charge.

À ce rythme réaliste, aucun goulot d'étranglement. p95 sous 40 ms sur les trois routes.

3. Goulot d'étranglement identifié

Un résultat propre sous rythme réaliste ne suffit pas à conclure « tout va bien » : avec 0,5-2 s de réflexion entre requêtes, 100 VU n'ont jamais plus qu'une poignée de requêtes réellement simultanées en vol — ce test-là ne peut pas voir une ressource dont la limite de concurrence est basse. Pour vérifier, même scénario, réflexion ramenée à zéro (`-e MIN_THINK_TIME_SECONDS=0 -e MAX_THINK_TIME_SECONDS=0`, 45 s) :

Route	p95 rythme réaliste	p95 rafale (sans délai)	Facteur
`GET /api/recruiter/catalogue`	38 ms	425 ms	×11
`GET /api/profiles/:id`	17 ms	750 ms	×45
`GET /api/profiles/:id/videos`	12 ms	438 ms	×36

Cause confirmée, pas seulement supposée. `packages/api/src/config/prisma.ts` construisait son pool PostgreSQL sans option `max` : `pg.Pool` retombe alors sur son défaut, 10 connexions. Échantillonné `pg_stat_activity` (par le même chemin de connexion que l'API, pas `docker compose exec` qui atteint une autre instance sur cette machine) à trois secondes d'intervalle pendant la rafale — chaque relevé totalise 11 connexions : le pool de l'API (10) plus la connexion de mesure elle-même.

```
12:23:31.913Z  active=1  idle=6   idle-in-transaction=4    → 11 au total
12:23:36.362Z  active=2  idle=4   idle-in-transaction=5    → 11 au total
12:23:40.754Z  active=1  idle=3   idle-in-transaction=7    → 11 au total
12:23:45.142Z  active=1  idle=4   idle-in-transaction=6    → 11 au total
```

Aucune requête n'est devenue plus lente à exécuter : elles attendent une connexion disponible avant même de commencer. `Profile.updatedAt`, `moderationStatus`, `certificationStatus`, `certificationScore` ont déjà leurs index (`schema.prisma`) — ce n'était donc pas la piste, comme anticipé.

4. Correctif appliqué — avant / après

Ajout de `DATABASE_POOL_MAX` (`packages/api/src/config/env.ts`, défaut `20`), passé au pool `pg` (`packages/api/src/config/prisma.ts`). 20 plutôt qu'une valeur plus haute : `max_connections` de PostgreSQL est 100 par défaut, partagé avec pgAdmin, les scripts de migration/seed, et une marge pour un second processus API.

Même scénario en rafale, avant/après, seul `DATABASE_POOL_MAX` change :

Route	p95 avant (pool = 10)	p95 après (pool = 20)	Amélioration
`GET /api/recruiter/catalogue`	425 ms	369 ms	-13 %
`GET /api/profiles/:id`	750 ms	608 ms	-19 %
`GET /api/profiles/:id/videos`	438 ms	361 ms	-18 %

Sorties brutes : `load-test/results/before-pool-fix-2026-09-08.{txt,json}` et `after-pool-fix-2026-09-08.{txt,json}`.

C'est une amélioration réelle et mesurée — pas une résolution complète, et ce rapport ne la présente pas comme telle. Le même échantillonnage `pg_stat_activity` montre le pool encore saturé après correctif : chaque relevé totalise 21 connexions (20 + la sonde) pendant la rafale, exactement comme il totalisait 11 avant. Le mur a reculé, il n'a pas disparu. Détail complet, y compris un essai exploratoire à 50 connexions et une piste non résolue (des connexions `idle in transaction` observées alors qu'aucune des trois routes testées n'ouvre de transaction applicative) : `load-test/results/pool-fix-2026-09-08.md`.

Le scénario réaliste (§2) n'a jamais eu besoin de ce correctif : il était déjà en dessous de 40 ms de p95 avec un pool de 10. Ce correctif durcit l'application contre un pic de trafic peu espacé dans le temps, pas contre le scénario « 100 recruteurs qui parcourent le catalogue » tel que littéralement demandé — qui, mesuré, n'a jamais posé de problème.

5. Ce qui reste ouvert

- Le pool de connexions reste la limite à 20 sous rafale extrême (§4) — une valeur plus haute améliore encore `profile_detail` / `profile_videos` mais pas `catalogue`, qui semble borné par autre chose (coût de la requête elle-même, à confirmer).
- Le motif de connexions `idle in transaction` sur des routes de lecture pure n'est pas expliqué — à vérifier côté comportement de l'adaptateur Prisma 7.
- Le défaut de visibilité §2 (candidat `PENDING` visible au catalogue, 404 à l'ouverture) est un défaut produit, pas un résultat de charge — à traiter séparément.

6. Pièces jointes

Tout est versionné dans `load-test/results/`, jamais reformaté après capture (voir `.prettierignore`) :

- `baseline-2026-09-08.{txt,json}` — scénario réaliste, §2

- `before-pool-fix-2026-09-08.{txt,json}` / `after-pool-fix-2026-09-08.{txt,json}` — rafale avant/après correctif, §3-4
- `findings-2026-09-08.md` / `pool-fix-2026-09-08.md` — le détail technique complet derrière ce rapport
- `load-test/README.md` — comment rejouer chacune de ces mesures